

Adaboost Algorithm

Weak Learners: a weak learner is a model that performs slightly better than random guessing.

It's a hypothesis (model) that has an error rate just less than 50% for binary classification.

* Any model which accuracy is a little more than 50%.

Ex: Small decision tree.

Decision Stumps: the simplest type of decision tree. It consists of only one split on a single feature.

Depth: 1 root with 2 child

Note For binary classification 2 class is -1 and +1 in adaboost not 0 and 1.

Stage-wise Additive Method: a strategy used in ensemble learning particularly in boosting algorithms where a strong learner is built incrementally, by adding one weak learner at a time.

Each added model (on stage) improves upon the previous one.

$$F(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

$h_m(x) \rightarrow$ the weak learner at stage/model m
α_m : The weight of the learner
$F(x)$: final ensemble model

each $h(x)$ is trained sequentially not all at once.

$$\boxed{\text{Error Rate} = 1 - \text{accuracy}}$$

After learning, the final classifier looks...

$$g(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right)$$

If sign came -ve $\rightarrow$ -ve class

If " " " +ve $\rightarrow$ +ve class

Ada-boost step by step

① Give an initial weight to each row
initial weight will be same for all.
initial row weight = $\frac{1}{n}$

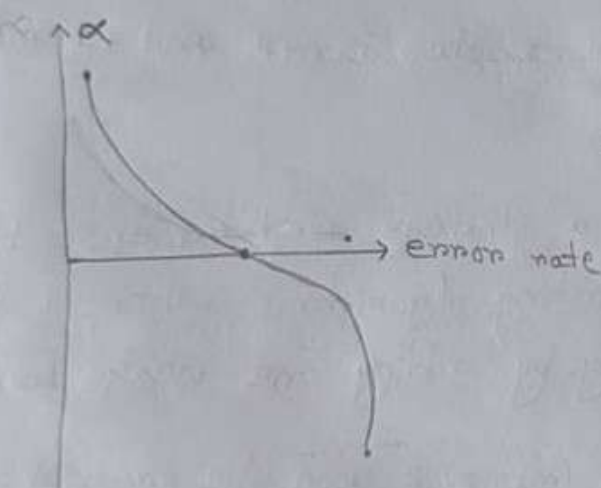
x_1	x_2	y	weight
3	7	1	0.2
2	9	0	0.2
1	4	1	0.2
9	8	0	0.2
3	7	0	0.2

Stage 1 train a decision tree on the dataset [depth will be 1] and sample weight with current weight of rows

$\rightarrow$ let's say tree will be m_1

$\rightarrow$ find $\hat{g}$ using the tree m_1 for all rows

$\rightarrow$ find the weight α for m_1



depends on error rate.

If $er \uparrow \alpha \downarrow$
If $er \downarrow \alpha \uparrow$

$$\boxed{\alpha = \frac{1}{2} \log \left(\frac{1 - \text{error}}{\text{error}} \right)}$$

error = sum of row weights where class is misclassified

$$\text{error} = \frac{\sum_{i=1}^n w_i \times 1[y_i \neq \hat{y}_i]}{\sum_{i=1}^n w_i}$$

ER $\rightarrow m_1 \rightarrow 0\% \rightarrow$ always right
 $\rightarrow m_2 \rightarrow 100\% \rightarrow$ always wrong

$m_3 \rightarrow 55\%$

Then m_1 and m_2 is more reliable

* more reliable
more alpha

→ a technique to increase the importance of misclassified rows/samples.

at one stage after computing the alpha, we have to up-sample these misclassified rows and need to send to the next stage.

→ increase the weight/boost the weight of misclassified rows.

decrease the weight of correct classified rows.

update weight

For misclassified:

$$\text{new weight} = \text{old weight} \times e^{\alpha_1}$$

For correct classified

$$\text{new weight} = \text{old weight} \times e^{-\alpha_1}$$

After updating all rows weight we have to normalize the weights so that

$$\text{Total weights} = \sum_{i=1}^n w_i \xrightarrow{\text{num of rows}} \text{becomes } 1$$

All rows weight normalized form

$$\text{weight} = \frac{\text{weight}}{\text{Total weight}}$$

Now do the up-sampling

① create weight range for all rows

range = Prev high bound to cumulative sum of weights till current row.

② generate n random numbers between 0 to 1 where n is the number of rows in current stage.

③ now take each number and check at which rows range the number lies and take the row to the new dataset

In this way we will make a new dataset for next stage.

In this way we will make m numbers of decision stamps along with m alphas.

and prediction will be

$$p(x) = \text{sign} \left(\sum_{i=1}^m \alpha_i h_i(x) \right)$$

Stamp: a tree with just one node
↑
weak learning

Difference between Bagging and Boosting

Bagging

- all model trains independently
- train on sample bootstrap
- all model has equal weight in voting
- lower overfitting risk

use case

- base model high variance low bias
- when we want to reduce overfitting
- noisy or small dataset

Boosting

- model train sequentially, correcting previous one.
- every model train on entire dataset with updated weight.
- different voting weight based on accuracy. accuracy ↑ weight ↑
- higher overfitting risk

use case

- base model is simple (underfitting)
- when we want to reduce bias and improve accuracy.
- large dataset. and want to get state-of-art performance
↑
highest level of performance currently achievable.